Web Development using PHP

Introducing PHP together with HTML

(Lecture 2)

Prepared By: BITS IT Team

Venue: GS Training Room

Table of Contents

1. PHP Block 4
Example 1.1 4
1.1 Comments in PHP 5
2. Declaring Variables in PHP 5
2.1 Naming Rules for Variables 6
Example 2.1 6
2.2 The Concatenation Operator 6
3. HTML Form handling using PHP 7
Example 3.1 7
3.1 The $_GET 7
3.1.1 When to use method="get"? 7
3.2 The $_POST 8
3.2.1 When to use method="post"? 8
Example 3.2 8
Example 3.3 8
3.3 The PHP $_REQUEST 9
Example 3.4 9
4. PHP Sessions 9
4.1 PHP Session Variables 10
4.2 Starting a PHP Session 10
4.3 Storing a Session Variable 10
4.4 Destroying a Session 11
5. PHP Operators 11
5.1 Arithmetic Operators 12
5.2 Assignment Operators 12
5.3 Comparison Operators 13
5.4 Logical Operators 13
6. PHP if else statement 13
6.1 The if Statement 14
Example 6.1 14

6.2 The if…else Statement 14
Example 6.2 15
6.3 The if…elseif….else Statement 15
Example 6.3 15
6.4 The Switch Statement 16
Example 6.4 16
7 Resources 18
7.1 websites 18
7.2 Books 18

1. PHP Block

A PHP scripting block always starts with **<?php** and ends with **?>**. A PHP scripting block can be placed anywhere in the document.

On servers with shorthand support enabled you can start a scripting block with <? and end with ?>.

For maximum compatibility, we recommend that you use the standard form (<?php) rather than the shorthand form.

```
<?php
?>
```

A PHP file normally contains HTML tags, just like an HTML file, and some PHP scripting code.

Each code line in PHP must end with a semicolon. The semicolon is a separator and is used to distinguish one set of instructions from another.

There are two basic statements to output text with PHP: **echo** and **print**. In the following example we'll be using **echo** statement to output the text "Hello World".

Create a web page (blank page) with index.php in your htdocs/training/**lecture2**/ directory.

Now Edit the file and Put the following code.

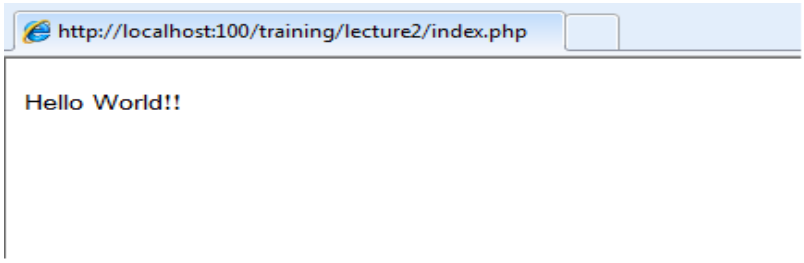
Example 1.1

```
<html>
<body>
<?php
   echo "Hello World";
?>
</body>
</html>
```

Note: The file must have a .php extension. If the file has .html extension, the PHP code will not be executed.

Run the page in the browser by typing http://localhost/training/lecture2/index.php

See what happens or displayed there.



1.1 Comments in PHP

In PHP, we use // to make a single-line comment or /* and */ to make a large comment block.

```
<html>
<body>
<?php
  //This is a comment
  /*
   This is
   a comment
   block
  */
?>
</body>
</html>
```

2. Declaring Variables in PHP

Variables are used for storing values, like text strings, numbers or arrays.

When a variable is declared, it can be used over and over again in your script.

All variables in PHP start with a $ sign symbol.

The correct way of declaring a variable in PHP:

$var_name = value;

Let's try creating a variable containing a string, and a variable containing a number:

```
<?php
  $txt="Hello World!";
  $x=16;
?>
```

In PHP, a variable does not need to be declared before adding a value to it.

In the example above, you see that you do not have to tell PHP which data type the variable is.

PHP automatically converts the variable to the correct data type, depending on its value.

In PHP, the variable is declared automatically when you use it.

2.1 Naming Rules for Variables

- A variable name must start with a letter or an underscore "_"
- A variable name can only contain alpha-numeric characters and underscores (a-z, A-Z, 0-9, and _)
- A variable name should not contain spaces. If a variable name is more than one word, it should be separated with an underscore ($my_string), or with capitalization ($myString)

Now open your index.php and add following (bold) text to it.

Example 2.1

```
<html>
<body>
<?php
   echo "Hello World";
   echo “<br>”;
   $txt="Hello World";
   echo $txt;
?>
</body>
</html>
```

The output of the code above will be:

```
Hello World
Hello World
```

Note the difference (can anybody explain it for others).

2.2 The Concatenation Operator

There is only one string operator in PHP.

The concatenation operator (.) is used to put two string values together.

To concatenate two string variables together, use the concatenation operator:

```
<?php
   $txt1="Hello World!";
   $txt2="What a nice day!";
   echo $txt1 . " " . $txt2;
?>
```

The output of the code above will be:

```
Hello World! What a nice day!
```

If we look at the code above you see that we used the concatenation operator two times. This is because we had to insert a third string (a space character), to separate the two strings.

3. HTML Form handling using PHP

When dealing with HTML forms it is to be noted that all forms element in an HTML page will **automatically** be available to your PHP scripts.

Example 3.1

Now create another file user.php and add following (bold) text to it.

This will contains an HTML form with two input fields and a submit button

```
<html>
<body>
<form action="welcome.php" method="post">
Name: <input type="text" name="fname" />
Age: <input type="text" name="age" />
<input type="submit" />
</form>
</body>
</html>
```

When a user fills out the form above and click on the submit button, the form data is sent to a PHP file, called "user_reply.php".

3.1 The $_GET

The built-in $_GET is used to collect values from a form sent with method="get".

Information sent from a form with the GET method is visible to everyone (it will be displayed in the browser's address bar) and has limits on the amount of information to send.

```
http://localhost:100/training/lecture2/user_reply.php?name=test&gender=Male&education=Masters&submit=Submit
```

3.1.1 When to use method="get"?

When using method="get" in HTML forms, all variable names and values are displayed in the URL.

Note: This method should not be used when sending passwords or other sensitive information!

However, because the variables are displayed in the URL, it is possible to bookmark the page. This can be useful in some cases.

Note: The get method is not suitable for very large variable values. It should not be used with values exceeding 2000 characters.

3.2 The $_POST

The built-in $_POST is used to collect values from a form sent with method="post".

Information sent from a form with the POST method is invisible to others and has no limits on the amount of information to send.

```
http://localhost:100/training/lecture2/user_reply.php
```

Note: However, there is an 8 Mb max size for the POST method, by default (can be changed by setting the post_max_size in the php.ini file).

3.2.1 When to use method="post"?

Information sent from a form with the POST method is invisible to others and has no limits on the amount of information to send.

However, because the variables are not displayed in the URL, it is not possible to bookmark the page.

Example 3.2

Now create another file user_reply.php and add following (bold) text to it.

```
<html>
<body>
Welcome <?php echo $_POST["fname"]; ?>!<br />
You are <?php echo $_POST["age"]; ?> years old.
</body>
</html>
```

Output could be something like this:

```
Welcome [the name which user provided]
You are [the age which user provided] years old.
```

Example 3.3

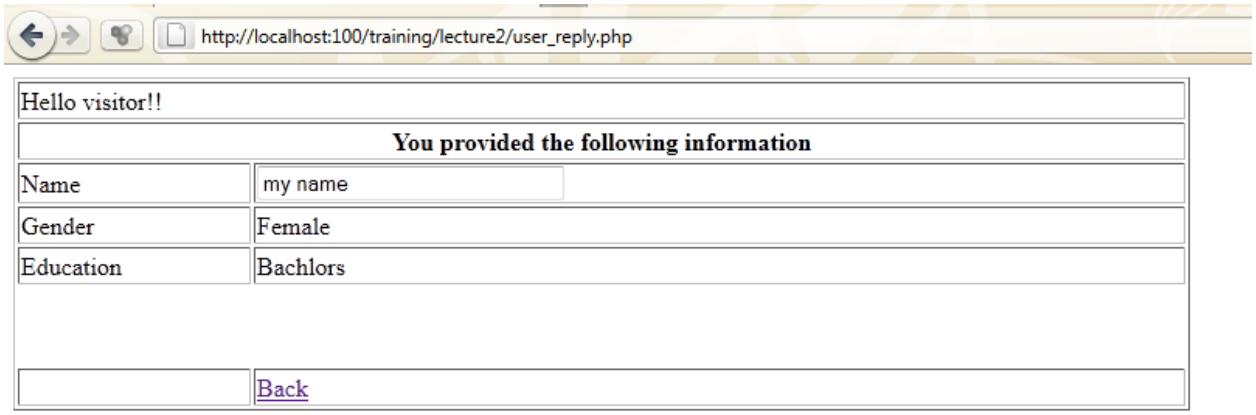
Using short output notation (change the text in user_reply.php file as follows).

```
<html>
```

```
<body>
Welcome <?=$_POST["fname"];?>!<br />
You are <?= $_POST["age"];?> years old.
</body>
</html>
```

The output will be same. Note the difference here.

Now open up your last assignment and display its user input as showed follows;



3.3 The PHP $_REQUEST

The PHP built-in $_REQUEST contains the contents of both $_GET, $_POST, and $_COOKIE.

The $_REQUEST function can be used to collect form data sent with both the GET and POST methods.

For example, Example 3.3 output can also be achieved as follows;

Example 3.4

```
<html>
<body>
Welcome <?=$_REQUEST["fname"];?>!<br />
You are <?= $_REQUEST["age"];?> years old.
</body>
</html>
```

4. PHP Sessions

A PHP session variable is used to store information about, or change settings for a user session. Session variables hold information about one single user, and are available to all pages in one application.

4.1 PHP Session Variables

When you are working with an application, you open it, do some changes and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are and what you do because the HTTP address doesn't maintain state.

A PHP session solves this problem by allowing you to store user information on the server for later use (i.e. username, shopping items, etc). However, session information is temporary and will be deleted after the user has left the website. If you need a permanent storage you may want to store the data in a database.

Sessions work by creating a unique id (UID) for each visitor and store variables based on this UID. The UID is either stored in a cookie or is propagated in the URL.

4.2 Starting a PHP Session

Before you can store user information in your PHP session, you must first start up the session.

Note: The session_start() function must appear BEFORE the <html> tag:

```
<?php session_start(); ?>

<html>
<body>

</body>
</html>
```

The code above will register the user's session with the server, allow you to start saving user information, and assign a UID for that user's session.

4.3 Storing a Session Variable

The correct way to store and retrieve session variables is to use the PHP $_SESSION variable:

```
<?php
   session_start();
   // store session data
   $_SESSION['views']=1;
?>
<html>
<body>
<?php
   //retrieve session data
   echo "Pageviews=". $_SESSION['views'];
```

```
?>
</body>
</html>
```

Output:

```
Pageviews=1
```

In the example below, we create a simple page-views counter. The isset() function checks if the "views" variable has already been set. If "views" has been set, we can increment our counter. If "views" doesn't exist, we create a "views" variable, and set it to 1:

```
<?php
   session_start();
   if(isset($_SESSION['views']))
   $_SESSION['views']=$_SESSION['views']+1;
   else
   $_SESSION['views']=1;
   echo "Views=". $_SESSION['views'];
?>
```

4.4 Destroying a Session

If you wish to delete some session data, you can use the unset() or the session_destroy() function.

The unset() function is used to free the specified session variable:

```
<?php
   unset($_SESSION['views']);
?>
```

You can also completely destroy the session by calling the session_destroy() function:

```
<?php
   session_destroy();
?>
```

Note: session_destroy() will reset your session and you will lose all your stored session data.

5. PHP Operators

Operators are used to operate on values.

The following tables list the different operators used in PHP.

5.1 Arithmetic Operators

Operator	Description	Example	Result
+	Addition	x=2 x+2	4
-	Subtraction	x=2 5-x	3
*	Multiplication	x=4 x*5	20
/	Division	15/5 5/2	3 2.5
%	Modulus (division remainder)	5%2 10%8 10%2	1 2 0
++	Increment	x=5 x++	x=6
--	Decrement	x=5 x--	x=4

5.2 Assignment Operators

Operator	Example	Is The Same As
=	x=y	x=y
+=	x+=y	x=x+y
-=	x-=y	x=x-y
*=	x*=y	x=x*y
/=	x/=y	x=x/y
.=	x.=y	x=x.y
%=	x%=y	x=x%y

5.3 Comparison Operators

Operator	Description	Example
==	is equal to	5==8 returns false
!=	is not equal	5!=8 returns true
<>	is not equal	5<>8 returns true
>	is greater than	5>8 returns false
<	is less than	5<8 returns true
>=	is greater than or equal to	5>=8 returns false
<=	is less than or equal to	5<=8 returns true

5.4 Logical Operators

Operator	Description	Example
&&	and	x=6 y=3 (x < 10 && y > 1) returns true
||	or	x=6 y=3 (x==5 || y==5) returns false
!	not	x=6 y=3 !(x==y) returns true

6. PHP if else statement

Conditional statements are used to perform different actions based on different conditions.

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

In PHP we have the following conditional statements:

- **if statement** - use this statement to execute some code only if a specified condition is true
- **if...else statement** - use this statement to execute some code if a condition is true and another code if the condition is false
- **if...elseif....else statement** - use this statement to select one of several blocks of code to be executed
- **switch statement** - use this statement to select one of many blocks of code to be executed

6.1 The if Statement

Use the if statement to execute some code only if a specified condition is true.

Syntax

if (*condition*) {*code to be executed if condition is true;}*

The following example will output "Have a nice weekend!" if the current day is Friday:

Example 6.1

```
<html>
<body>
<?php
   $d=date("D");
   if ($d=="Wed")
   {
    echo "Have a nice weekend!";
   }
?>
</body>
</html>
```

Notice that there is no ..else.. in this syntax. The code is executed **only if the specified condition is true**.

6.2 The if...else Statement

Use the if....else statement to execute some code if a condition is true and another code if a condition is false.

Syntax

if (*condition*)
 {

code to be executed if condition is true;
}
else
{ *code to be executed if condition is false;* }

Example 6.2

The following example will output "Have a nice weekend!" if the current day is Wednesday, otherwise it will output "Have a nice day!":

```
<html>
<body>
<?php
   $d=date("D");
   if ($d=="Wed")
   {
     echo "Have a nice weekend!";
   }
   else
   { echo "Have a nice day!";}

?>
</body>
</html>
```

6.3 The if...elseif....else Statement

Use the if....elseif...else statement to select one of several blocks of code to be executed.

Syntax

if (*condition*)
 {*code to be executed if condition is true;}*
elseif (*condition*)
 {*code to be executed if condition is true;}*
else
 {*code to be executed if condition is false;* }

Example 6.3

The following example will output "Have a nice weekend!" if the current day is Wednesday, and "Have a nice Saturday!" if the current day is Saturday. Otherwise it will output "Have a nice day!":

```
<html>
```

```
<body>
<?php
$d=date("D");
if ($d=="Wed")
 {echo "Have a nice weekend!";}
elseif ($d=="Sat")
 {echo "Have a nice Saturday!";}
else
 {echo "Have a nice day!";}
?>
</body>
</html>
```

6.4 The Switch Statement

Conditional statements are used to perform different actions based on different conditions.

Use the switch statement to select one of many blocks of code to be executed.

Syntax

switch (*n*)
{
case *label1:*
 code to be executed if n=label1;
 break;
case *label2:*
 code to be executed if n=label2;
 break;
default:
 code to be executed if n is different from both label1 and label2;
}

This is how it works: First we have a single expression *n* (most often a variable), that is evaluated once. The value of the expression is then compared with the values for each case in the structure. If there is a match, the block of code associated with that case is executed. Use **break** to prevent the code from running into the next case automatically. The default statement is used if no match is found.

Example 6.4

```
<html>
<body>
<?php
switch ($x)
```

```
{
case 1:
 echo "Number 1";
 break;
case 2:
 echo "Number 2";
 break;
case 3:
 echo "Number 3";
 break;
default:
 echo "No number between 1 and 3";
}
?>
</body>
</html>
```

7 Resources

7.1 websites

http://www.w3schools.com/

7.2 Books

Professional PHP Programming by Wrox word press Ltd. **ISBN:** 81-7366-201-0 (chapter 1- 6)